| | |
|---|---|
| **Team** | Crystal |
| **Members** | - Anderson Corriero<br>- Andrew Ebinger<br>- Jacob Lizak<br>- James Bryceland<br>- Sal Ferrara<br>- Talon Carsto |
| **Assignment** | Lab 5 |

# Highlights

- Connect to Raspbot
    - Using app
    - Using JupyterLab
    - Using VNC viewer
- Test object detection and avoidance
- Test gesture identification and control

# Encountered Problems with Solutions

## Problems Summary with Solution

1. Had an issue where the software would not recognize the camera because its index was out of range.

## Debug Procedure

1. We tried using the command to kill the processes from the phone app, and going into the file itself to change the index counter from “0” to “-1”, to no avail. We ultimately could not find what the issue was, but found that it ran correctly after power cycling.

# Execution of Lab Tasks

## Task 1

Task 1 had us connect to the robot and test basic functionality. To start, we connected to the robot’s network and used the Yahboom app to control the car. This was a very simply way to test the functionality of key components, such as the motors and the camera. It also allowed the team to quickly learn the bot's features and begin formalizing a plan for the final project. The team tested most of the built-in modes, but the color detection mode really stood out to us; we spent a good ten minutes showing it different colors and seeing what it detected. This could be useful in situations where identical boxes are color-coded based on send location, allowing a robot to sort packages quickly. The team also tested the drive motors and wheels, as shown in

Figure 1. During testing, we found that while straphing was effective for navigating tight areas, motor torque caused one or more wheels to slip, initially drifting the bot off course. This will definitely be something the team will have to consider as we move towards the final project.

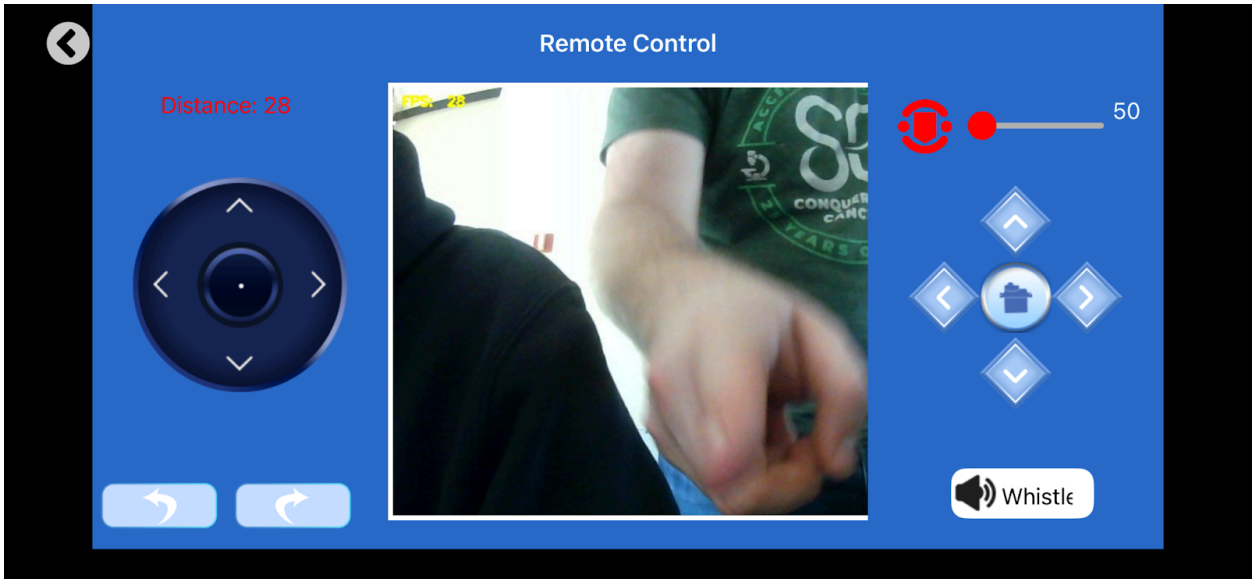



Figure 1: remote control mode used in testing motors

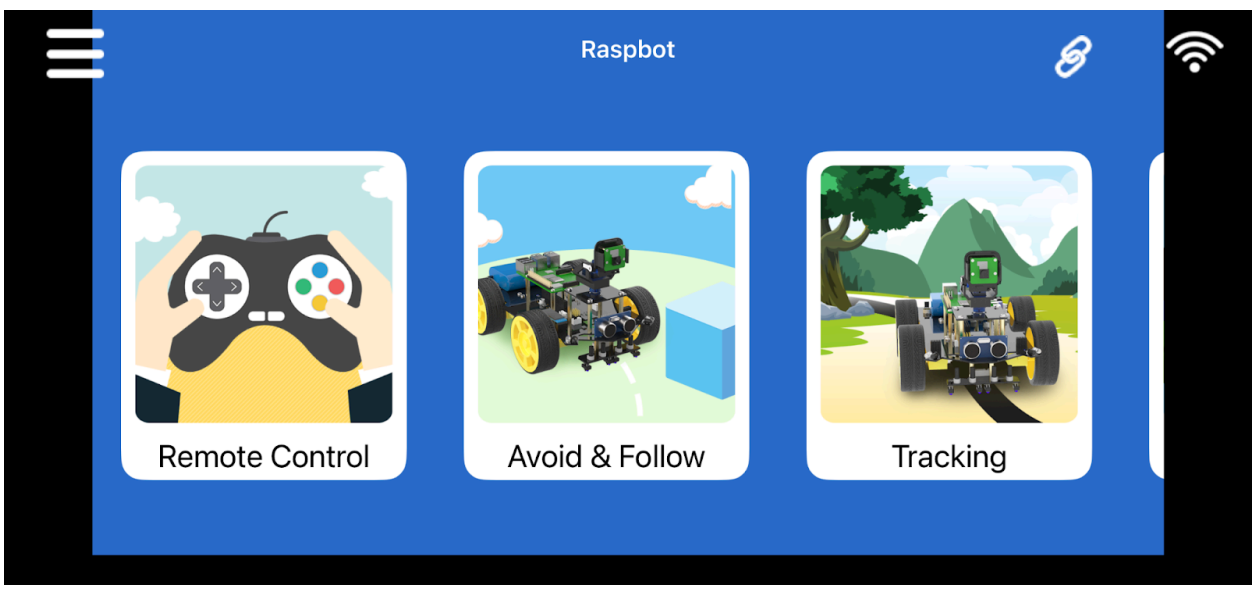



Figure 2: Home menu of Yahboom app

Next, we connected to the robot's network and used its IP address to access the robot via Jupyter Lab. After connecting to the robot's network on a laptop and inputting the secure password, we moved into a browser and input http://<robot_ip>:8888/lab. Since Jupyter is a browser-based coding environment, this would allow us to connect and control the robot over its built-in network. We tested moving the motors using the "5.Motor control.ipynb", this allowed us to control each motor individualy using sliders. Seen in Figure 3 below, the first function defines

and assigns a value to each motor dependent upon the user input value from the second function.

```
# 控制电机运动 Control motor movement
def run_motor(M1,M2,M3,M4):
    bot.Ctrl_Muto(0, M1)
    bot.Ctrl_Muto(1, M2)
    bot.Ctrl_Muto(2, M3)
    bot.Ctrl_Muto(3, M4)
    return True


# 创建四个滑块来控制电机 Create four sliders to control the motor
interact(run_motor, \
         M1=widgets.IntSlider(min=-255,max=255,step=1,value=0), \
         M2=widgets.IntSlider(min=-255,max=255,step=1,value=0), \
         M3=widgets.IntSlider(min=-255,max=255,step=1,value=0), \
         M4=widgets.IntSlider(min=-255,max=255,step=1,value=0));
```

Figure 3: Motor control code

The Raspbot V2 comes pre-installed with VNC viewer, which allows us to connect and use the robot, given that we use the same network. With VNC viewer installed on our computer, while making sure to use the same network as the robot, we once again connected to the robot using its ip. Within VNC viewer, we used source docker_ros2.s to enter into the docker (see Fig. ). One important part of the docker is that all terminals must run within the docker container. This was done by running docker ps -a in a new terminal, which gave us a list of containers with corresponding IDs. Taking the most recent of these containers, we then ran docker exec -it <ID> /bin/bash, where <ID> was replaced with the ID of the most recent container. Finally, we created a team folder inside /home/pi/temp.

With our team folder set up, we then started controlling the robot. This was done by running the driver node: ros2 run yahboomcar_bringup Mcnamu_driver, and in a separate terminal, ran the teleop node: ros2 run yahboomcar_ctrl yahboom_keyboard. The robot was now controllable with our keyboard. Using the keyboard, we where able to control the robot and collect positioning data based on the robot's orientation within the environment. This data showed us the robot used a Cartesian system, with positions made up of a X, a Y, and a Z. After running rqt_graph to visualize the node graph (See Fig. ), we ran exit in the original terminal to shut down the docker.

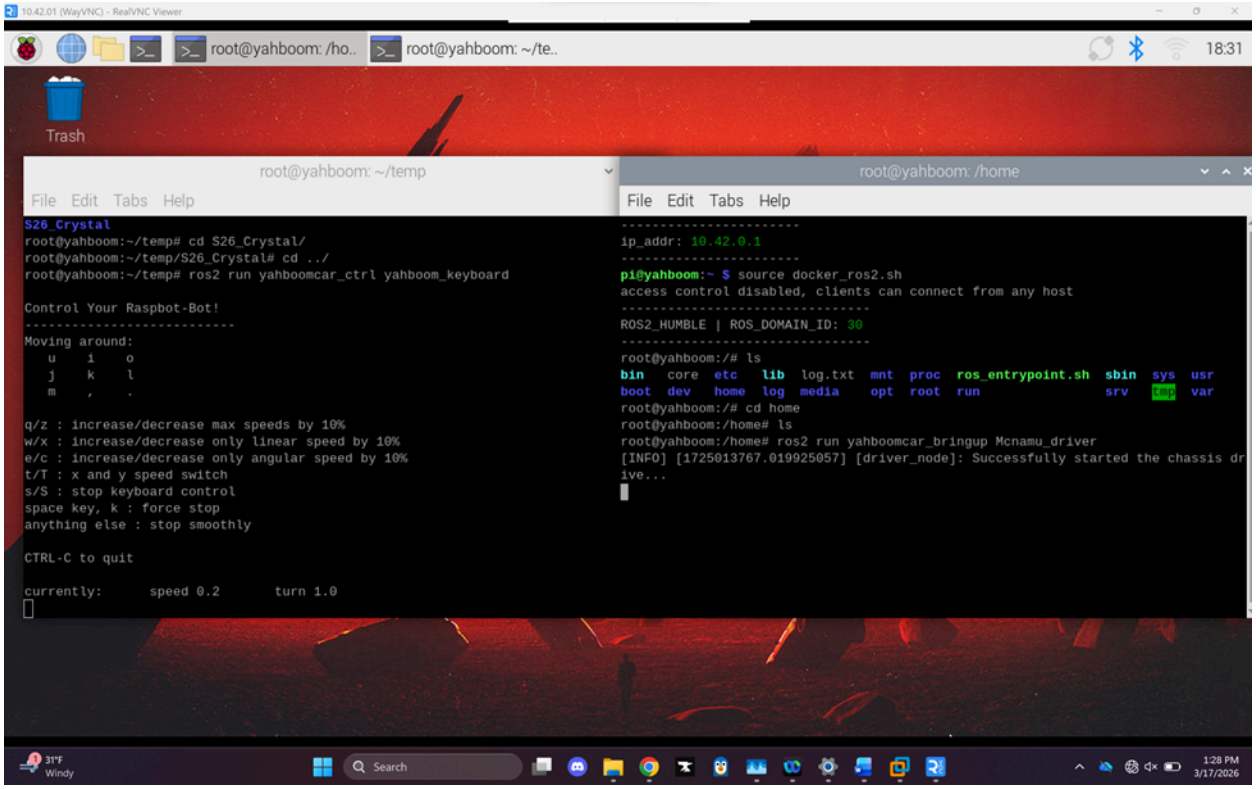
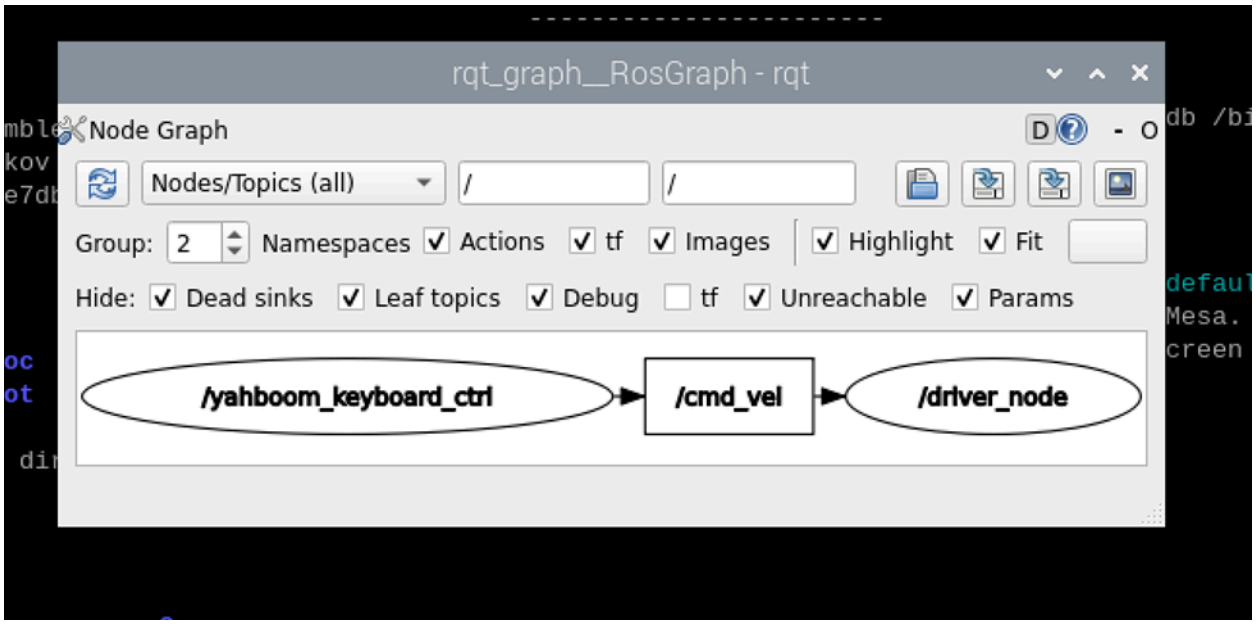



Figure : Terminals showing entering the docker and and running the driver node along with the keyboard control node




rqt graph showing how the keyboard control node and the driver node interact via the cmd_vel topic

## Task 2

For task 2, we used the onboard ultrasonic sensor to explore obstacle avoidance. Within project_demo/Comprehensive_gameplay/ultrasonic_obstacle_avoidance in Jupyter Lab, we find the obstacle avoidance code. With our file open and our robot connected, we used the triangle play button to step through the code and test the robot's obstacle avoidance.

The ultrasonic sensor of Raspbot V2 has several advantages over alternatives like IR sensors or cameras. Firstly, ultrasonic sensors are unaffected by light, meaning they can work in complete darkness and don't get unintended readings from light changes. They also don't depend on color unlike IR sensors. There are also of course some disadvantages. Notably, they are less accurate than IR sensors. Not only are long distances more of a challenge, but so are the physical characteristics of the surfaces it detects. If the surface has a lot of texture or is angled, the sound can be deflected and dampened by the material. Because of this, they are also worse at mapping the shapes of objects. Despite these disadvantages, our basic testing showed the robot was relatively successful at detecting and avoiding obstacles. This is mostly since much of our testing didn't count on the disadvantages of the ultrasonic sensor. For future testing in path planning and avoidance, we can keep the ultrasonic sensor's disadvantages in mind. We can make sure to use flat, solid surfaces that sit perpendicular to the floor.

## Task 3

Task 3 involved engaging with the RGBD camera to use the hand gesture tracking software preloaded onto our robot. Firstly, we engaged with the robot using VNC Viewer, which pulled up a linux machine terminal similar to the virtual machine used in previous labs. This machine already contained our teams file in the temp folder, so no new directory had to be made. From there we opened two terminals, using one of them to engage the docker container and the other to enter it. This docker container was different from the container used in step 1, since after closing a docker container it cannot be opened again.

In the first terminal, we ran `ros2 launch yahboomcar_bringup bringup.launch.py`, which prepped the motor controllers on the robot to receive commands from some input, usually the keyboard control package in our case. In the other terminal, we ran `ros2 run yahboomcar_mediapipe FingerCtrl`, which ran the camera and started the gesture recognition software. The hand gesture software identified various points on a shape and identified it to be a human hand, which it then applied a wireframe to. Then, it tracked the shape that the wireframe took and compared that to known shapes (gestures), corresponding certain gestures with movement actions to be taken by the robot. We tested the following:

1. The "Rock" gesture (seen in figure _), which caused the robot to move in an S shape.
2. The "Thumbs down" gesture, which caused the robot to move forward.
3. The "OK" gesture (a clenched fist with a thumb pointing off to the side), which caused the robot to move in a circle.
4. The "Yes" gesture (or "peace sign") which caused the robot to move in a square

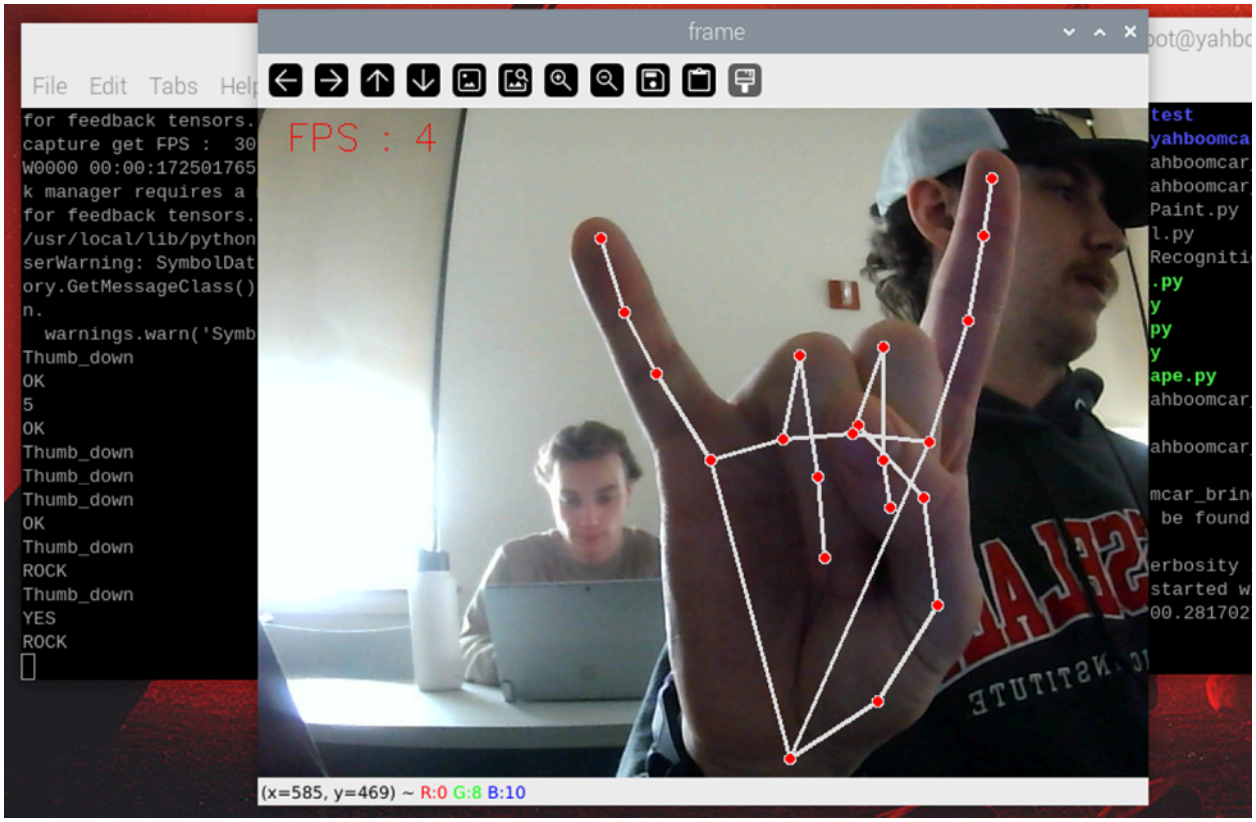



***Figure_**: Gesture recognition software applying a wireframe to a hand in frame and recognizing a gesture through a text output in the left terminal*

While these tests were happening, our group monitored the rqt graph for these nodes. Contrary to previous graphs, the data path for the information being passed through the system was much less intuitive. Where we would have expected it to be fairly linear with branches feeding data into the main line, the graph seemed to indicate that the majority of the data processing was done in the final node. The nodes included `/posectrlarm`, `/mediapipe` and `/drivernode` and the topics included `/servo`, `/buzzer`, and `/cmd_vel`. Initially we were confused as to why the mediapipe node published to the cmd_vel topic instead of

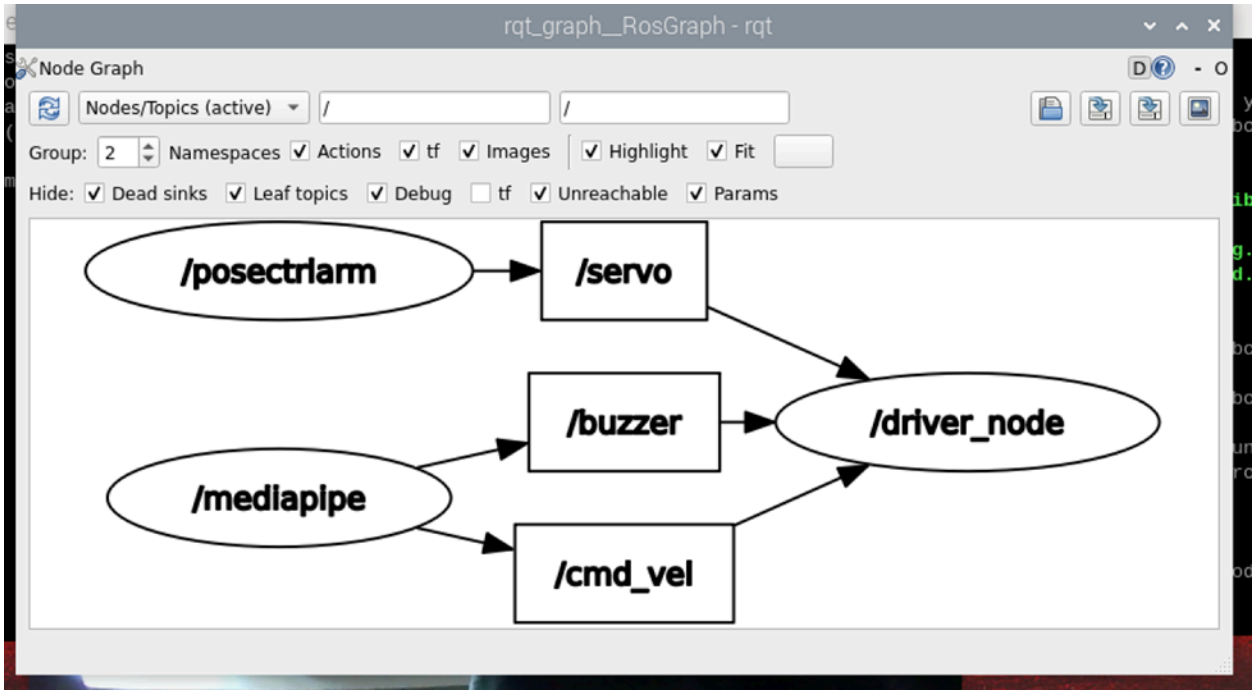



***Figure_**: RQT graph generated while the gesture recognition software was active.*

## Future

This lab served as a strong introduction to the Raspbot and the different methods available to control it. As mentioned previously, the team plans to closely monitor any wheel slip when the robot starts from rest and evaluate whether it presents a significant issue. If necessary, a potential solution may involve adjusting the initial motor input values to enable a smoother startup. Although we did not extensively modify motor parameters during this lab, this presents an opportunity for further optimization in future work.

Looking ahead, the team is particularly interested in integrating the camera and ultrasonic sensor to enable object detection based on both proximity and color. This combination could support more advanced autonomous behaviors and is a promising direction for our final project. Additionally, we are excited to continue building on the skills developed in this lab, especially as we transition into more hardware-focused applications where these concepts can be applied in a more comprehensive and practical context.

## Collaboration Summary

During this lab, the team collaborated extensively to complete each task efficiently. Because we only had access to a single robot, coordination was essential. For every task, we designated one “point person” responsible for directly interfacing with the robot. This approach prevented multiple users from attempting control simultaneously, which could have caused

conflicts or errors. Supporting this role, two additional team members followed along with the instructions to provide guidance and catch any mistakes, while others focused on documenting progress and troubleshooting issues as they arose.

Task One involved the entire team, as it required exploring three different methods of interfacing with the robot. Jacob Lizak and Anderson Corriero were responsible for connecting to and testing the Yahboom app, allowing us to better understand the robot's capabilities for future tasks. Sal Ferrara and Talon Carsto handled interfacing through the Jupyter Notebook, while Andrew Ebinger and James Bryceland set up the VNC viewer and verified connectivity. Since these subtasks were relatively straightforward, dividing them among smaller groups of two proved to be an efficient strategy.

For Task Two, Sal Ferrara served as the primary operator, supported by Talon Carsto and Anderson Corriero. Task Three was led by Andrew Ebinger, with assistance from Jacob Lizak and James Bryceland. Moving forward, the team plans to rotate primary responsibilities to ensure that all members gain hands-on experience interfacing with the robot.

# References

[1] Oakes, K (2026) Intro to Robot Programming. Rensselaer Polytechnic Institute